

The Invisible Gap: Absence Blindness in Large Language Models and the Case for Externalized Presence

Andres G

Independent Research

Pembroke Pines, Florida, USA

marksbenjamin256@gmail.com

Abstract—Large language models (LLMs) are trained to predict the next token given a preceding context. This objective, however efficient, encodes a fundamental epistemic asymmetry: what is *present* in the context window exerts probabilistic force on the next token, while what is *absent*—a missing deliverable, an unverified claim, an unchecked task item—exerts zero force. We term this property absence blindness: the model cannot perceive, and therefore cannot act upon, the gap. This paper formalizes the absence blindness hypothesis, surveys the empirical literature that corroborates it across hallucination, omission, and self-correction failure modes, and proposes the *Externalized Presence* design principle: gaps must be reified into explicit context objects before they can influence generation. We demonstrate that every successful intervention on LLM incompleteness—retrieval-augmented generation, checklist injection, verification agents, tool-result grounding—can be understood as an instance of Externalized Presence. We conclude with a taxonomy of gap types and a practical scaffold specification for agentic systems that operationalizes this principle at inference time.

Index Terms—large language models, absence blindness, hallucination, omission, externalized presence, agentic scaffolding, next-token prediction, verification

I. INTRODUCTION

The dominant paradigm for training large language models is *next-token prediction* (NTP): given a sequence of tokens $x_1, x_2, \dots, x_{t-1}$, the model learns to assign a probability distribution over the vocabulary for position t [1]. This objective is both remarkably powerful and subtly treacherous. Its power lies in the compression of world knowledge into distributional patterns; its treachery lies in what it cannot see.

Consider a model tasked with writing a software specification that must include five sections. If the model generates four sections and moves on, nothing in the loss signal during training specifically penalizes the omission of the fifth—the omission leaves no token behind to predict, and thus generates no gradient. At inference time, the partially-completed output constitutes the context; the missing section is not in that context, and hence cannot exert probabilistic force on the next token. The gap is, from the model’s perspective, invisible.

This paper names this phenomenon **absence blindness** and argues that it is not a correctable bug but a *structural consequence* of the NTP objective. We then ask: if the gap is invisible from the inside, how can it be made visible from the outside? We call the answer **Externalized Presence**: the

practice of transforming gaps into explicit, present-in-context objects that can influence generation.

Our contributions are as follows:

- 1) A formal definition of absence blindness grounded in the information theory of autoregressive generation (§III).
- 2) A synthesis of the empirical literature demonstrating absence blindness across three failure modes: hallucination, omission, and self-correction collapse (§IV).
- 3) The Externalized Presence principle, a unifying design framework explaining why RAG, checklist prompting, verification agents, and tool-use all work (§V).
- 4) A gap taxonomy and a practical scaffold specification for agentic systems (§VI–§VII).

II. BACKGROUND AND RELATED WORK

A. The Next-Token Prediction Paradigm

NTP has been studied as both the engine of emergent capability and the source of fundamental limitation. McCoy et al. [2] argue through their “teleological approach” that LLM behavior is best understood by the problem they are trained to solve: predicting probable next words in internet text. They identify *probability of the task*, *probability of the target output*, and *probability of the position* as three factors governing success and failure—a framing that implicitly captures absence blindness: rare or missing content is by definition low-probability, and thus suppressed during generation.

Bachmann and Nagarajan [3] crystallize the concern formally: they distinguish *autoregressive inference* from *teacher-forced training* and show that NTP can fail to learn accurate models of multi-step reasoning because the training signal at any given step does not account for downstream consequences. Crucially, their analysis exposes a myopia in the loss function: it rewards accurate next-token assignment, not completeness of semantic coverage.

Downes et al. [4] argue against the reductive view that “LLMs are just next-token predictors,” pointing to emergent representations that go beyond surface-level pattern matching. While we agree that LLMs are more than token-probability machines, this paper’s thesis is orthogonal: even granting emergent capabilities, the model still cannot perceive what is not present in context.

B. Hallucination and the Fabrication of Presence

Hallucination research has historically focused on the production of false content—tokens that are confidently generated but factually wrong. Banerjee et al. [5] demonstrate that hallucination is not merely a training artifact but a mathematical inevitability: for any sufficiently expressive model, there exist well-formed queries whose correct answers lie outside the model’s training distribution. Similarly, Xu et al. [6] formalize the impossibility of complete hallucination elimination. These results are consistent with absence blindness: a model that cannot perceive absence will generate probable-sounding content to fill the gap rather than pausing to acknowledge it.

Critically, hallucination and omission are dual manifestations of the same phenomenon. When the model encounters an absent fact, it either generates a plausible confabulation (commission) or skips over the gap entirely (omission). Both represent the invisible gap; they differ only in whether the model’s generation process fills the void with probable tokens or remains silent. Dejl et al. [7] formalize this complementarity, demonstrating that LLM-generated texts frequently omit key information in ways that are as harmful as outright factual inaccuracies, and proposing automated comprehensiveness metrics to detect such silences.

C. The Lost-in-the-Middle Effect

Liu et al. [8] provide a concrete empirical instance of absence blindness at the attention level: LLM performance degrades significantly when relevant information is placed in the *middle* of a long context, while information at the beginning and end receives preferential attention. Hsieh et al. [9] trace this to an intrinsic U-shaped positional attention bias. The implication is that *even present information can become effectively absent* if it occupies the wrong position—confirming that presence is a necessary but not sufficient condition for influence. Lu et al. [10] further reveal a “know but don’t tell” phenomenon: LLMs may encode the position of target information in hidden representations yet fail to surface it in generation, suggesting a disconnect between latent presence and generative influence.

III. FORMALIZING ABSENCE BLINDNESS

A. The Autoregressive Objective

Let $\mathcal{V}$ denote the vocabulary, $X = (x_1, \dots, x_T)$ a token sequence, and θ the model parameters. The NTP objective minimizes:

$$\mathcal{L}_{\text{NTP}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_1, \dots, x_{t-1})$$

Each term depends only on tokens $x_1, \dots, x_{t-1}$ that are *present* in the context window. Tokens or facts that do not appear in the prefix are not represented in the gradient signal.

B. The Absence Blindness Theorem (Informal)

Theorem (Informal). *For any well-formed task specification T requiring deliverable set $D = \{d_1, \dots, d_k\}$, the probability that the autoregressive model generates all $d_i \in D$ is bounded above by the probability of each d_i appearing in training contexts that condition on all prior $d_j, j < i$ having been generated. Any d_i whose antecedent generation pattern is low-frequency in the training corpus will be systematically suppressed at inference time, regardless of its task-required presence.*

This result follows directly from the distributional fidelity property of NTP: the model converges toward patterns that are probable under the training distribution. Missing deliverables, by definition, do not appear in training data as explicit tokens to be predicted, so no gradient reinforces their generation.

C. Three Modes of Invisible Gaps

We distinguish three operational manifestations of absence blindness:

- 1) **Deliverable gaps:** Required task outputs that were never generated (e.g., a missing section in a report, an unchecked constraint in code generation).
- 2) **Epistemic gaps:** Claims in the generated output that are unverified—the model does not “know” it doesn’t know, because the absence of grounding evidence generates no explicit signal.
- 3) **Contextual remainder gaps:** Earlier parts of a long conversation or document that have scrolled out of effective attention range, even if technically within the context window.

All three share the key property: *from the model’s perspective, the gap looks identical to a completed requirement*—both are represented by the absence of any token, and NTP assigns equal (near-zero) weight to both.

IV. EMPIRICAL EVIDENCE FOR ABSENCE BLINDNESS

A. Hallucination as Gap-Filling

Heyman and Zylberberg [11] demonstrate that reasoning LLMs systematically hallucinate graph edges not specified in prompts when solving graph-coloring problems. The model, encountering an absence of edge information, fills the gap with probable-sounding content rather than registering the absence as an epistemic signal. This behaviour—which they document across six frontier models including o1-mini, DeepSeek-R1, and Claude 3.7 Sonnet—is not corrected by chain-of-thought prompting, confirming that the gap is invisible even during extended reasoning.

Omar et al. [12] embed fabricated clinical details into prompts and find that LLMs elaborate on the false information in 50–82% of cases across six models. The adversarial fabrication *became present* in context; the model amplified it. The implicit control in their study—the same clinical fact absent from the prompt—did not trigger generation of the false detail. Presence controls generation; absence does not constrain it.

Yu et al. [13] trace a causal pathway from self-attention to hallucination: attention misdirection causes models to over-rely on parametric associations while ignoring contextual signals, creating a de facto absence of grounding even when evidence is technically present.

B. Omission as the Silent Dual of Hallucination

While hallucination inserts false content, omission deletes required content. Mess et al. [14] document this in clinical AI scribes: “errors of omission, fabrication, or substitution may occur,” with omissions being particularly insidious because they are silent—no token marks the location of the missing fact. Dejl et al. [7] quantify omission rates across popular open-weight LLMs and show that selective omission of key information is systematic, not random, and correlates with low training-frequency of the omitted fact.

Lee et al. [15] introduce the NOAH benchmark, which documents that LLMs systematically suppress narrative-incongruent events in video captioning. The model’s prior over probable story continuations overrides its obligation to report what actually appears, suppressing events that don’t fit the expected sequence. This is a direct demonstration of absence blindness in temporal reasoning.

C. Self-Correction Collapse

If absence blindness were merely a first-pass generation problem, self-correction mechanisms would cure it. The evidence suggests otherwise.

Kamoi et al. [16] conduct a comprehensive critical survey of self-correction strategies. Their central finding is stark: *no prior work demonstrates successful self-correction via prompting alone in tasks without reliable external feedback.* The explanation aligns with absence blindness: when asked to verify its own output, the model attends to the tokens it generated, not to the tokens absent from the required deliverable set. The gap remains invisible because it generates no token for the model to attend to.

Chen et al. [17] provide the clearest empirical demonstration. They present models with erroneous claims and vary only the “role” in which the claim appears—whether as the model’s own prior output (assistant role) or as an external message (user or tool role). Relabeling the claim from the assistant’s own output to an external source lifts explicit correction rates by 23 to 93 percentage points, a result significant at $p < 0.001$ across 13 model-domain cells. The content of the claim is byte-identical; only its source role changes. The conclusion is unambiguous: the model is not evaluating claim truth—it is responding to the *presence or absence of an externally-framed challenge*. Absence (no external challenge) produces no correction; presence (external framing) forces engagement.

Pan et al. [18] survey self-correction more broadly and reach a compatible conclusion: self-correction “works well in tasks that can use reliable external feedback” but fails when the model is left to evaluate its own output in isolation. External feedback is precisely the mechanism of Externalized Presence.

V. THE EXTERNALIZED PRESENCE PRINCIPLE

A. Definition

We define the **Externalized Presence Principle (EPP)** as follows:

Any gap that must influence the model’s next token must first be reified as an explicit, present-in-context object. A gap that remains unrepresented in the context window exerts zero generative force.

The EPP is not a novel empirical claim but a logical consequence of the NTP objective—the theoretical contribution of this paper is to name the principle clearly, trace its origins to the formalism, and show that the entire literature on LLM interventions converges on it.

B. Externalized Presence in Retrieval-Augmented Generation

RAG [19] is the most widely deployed instance of EPP. The gap addressed by RAG is epistemic: the model’s parametric knowledge does not contain current or specialized facts. Rather than hoping the model notices the gap, RAG *retrieves* external documents and injects them into the context window, converting the epistemic gap into present evidence.

Empirically, RAG reduces hallucination rates dramatically. Zhang [20] reports a reduction from 68% to 10% hallucination rates on open-domain QA with a retrieval-augmented framework. Wood and Forbes [21] demonstrate near-elimination of hallucination on the RAGTruth benchmark when retrieval is properly grounded. The mechanism is EPP in its purest form: a gap in parametric knowledge is converted into a present document passage.

C. Externalized Presence in Checklist and Verification Prompting

Liu et al. [22] introduce Questions-of-Thoughts (QoT), a scaffold that transforms an ordered list of engineering requirements into explicit self-questioning steps. Without QoT, deliverable gaps are invisible; with QoT, each unfulfilled requirement is reified as a pending question in the context, creating a present-in-context signal the model must address. They report consistent quality improvements on completeness metrics for larger models, with omission errors directly reduced by the scaffold.

The chain-of-verification approach [23] follows the same logic: after initial generation, the scaffold generates explicit verification questions from the output and retrieves evidence for each. The unverified claim—which was a gap from the model’s perspective—becomes a present question with an explicit answer slot. This converts the epistemic gap into a token-present constraint.

D. Externalized Presence in Tool-Use and Agentic Loops

Kim et al. [24] demonstrate that a Recursive Criticism and Improvement (RCI) approach enables LLM agents to execute computer tasks reliably—but only when the criticism step explicitly surfaces failures as natural language statements in context. The uncompleted step becomes a present assertion

(“the file was not saved”) rather than an absent token. Their work on MiniWoB++ shows that RCI outperforms supervised and reinforcement learning baselines precisely by making task incompleteness visible.

Sun et al. [25] study *silent tool failures*—cases where an agent’s tool call returns without explicit error but also without completing the required action. They find that without explicit failure signaling, LLM agents proceed as if the task succeeded. The silent failure is an absence; it exerts no force. Their proposed intervention is precisely EPP: the agent must be equipped with mechanisms that convert silent tool failures into explicit context objects.

E. Why Intrinsic Self-Correction Fails

The failure of intrinsic self-correction is the strongest evidence for absence blindness and the strongest argument for EPP. If the model could detect its own gaps, there would be no need for external scaffolds. But as Chen et al. [17] and Kamoi et al. [16] demonstrate, the model cannot—because the gap, by definition, generates no token to attend to.

This is not a failure of intelligence but of *attention physics*: self-attention mechanisms can only operate over tokens that are present in the sequence [8]. Qu et al. [26] train models via imitation learning to recursively detect and correct errors over multiple turns, finding that this succeeds only when the error is explicitly verbalized in a prior turn—i.e., made present. Renze and Guven [27] show that self-reflection improves problem-solving only in architectures where the reflection step explicitly generates a mistake token sequence, not merely where the model is instructed to “check its work.”

VI. A TAXONOMY OF GAP TYPES

Building on the three modes defined in §III and the empirical evidence of §IV, we propose a four-class taxonomy of gaps relevant to LLM systems:

TABLE I
TAXONOMY OF INVISIBLE GAPS AND THEIR EXTERNALIZATION STRATEGIES

Gap Type	Example	Externalization	Evidence
Deliverable gap	Missing section in a report	Checklist injection, QoT scaffold	[22]
Epistemic gap	Unverified factual claim	RAG retrieval, CoV prompting	[20]
Remainder gap	Earlier task step scrolled out of attention	Explicit status summary, memory injection	[8]
Failure gap	Silent tool error, unconfirmed execution	Tool result injection, failure assertion	[25]

Each gap type shares the structural property: from the model’s perspective it is indistinguishable from a completed requirement, because both are represented by zero tokens. Only an external system can classify the gap and produce the token representation that makes it actionable.

VII. SCAFFOLD SPECIFICATION FOR EXTERNALIZED PRESENCE

We specify a minimal *Gap-Hunter* scaffold that operationalizes EPP in agentic systems. The scaffold operates as an outer loop around LLM generation, executing three functions:

A. Gap Declaration

Before the model begins a task, the scaffold instantiates a *presence manifest*: an explicit, in-context enumeration of required deliverables, verified claims, and confirmed execution steps. Each element is represented as a token-present object with an explicit status field (PENDING | SATISFIED | FAILED).

B. Gap Detection

After each model generation step, the scaffold runs a *gap detector*—which may itself be an LLM, a rule-based extractor, or a structured comparison against the presence manifest. The detector is external to the generating model and evaluates whether each manifest item has been satisfied. Unsatisfied items are marked PENDING and returned to context.

C. Gap Injection

Unsatisfied items are injected into the next generation context as *gap objects*: structured token sequences that make the gap explicit. For example:

```
[GAP: Section "Risk Analysis"
not yet generated. Required
per task specification.]
```

This converts the deliverable gap into a present-in-context constraint that the model can now attend to and resolve. The cycle repeats until all manifest items are SATISFIED or a halting condition is met.

D. Relation to Existing Frameworks

This scaffold generalizes several existing approaches. AFlow [28] automates agentic workflow generation with iterative optimization loops that implicitly implement gap detection and re-injection. AgentSpec [29] specifies runtime enforcement constraints for LLM agents that function as gap declarations. The Quality Checker in QualityFlow [30] “imagines” whether synthesized programs satisfy test cases before allowing submission—an externalized evaluation of a potential gap. TaskGen [31] uses StrictJSON to enforce structured output from LLMs, converting schema violations (a form of gap) into explicit error tokens that trigger correction.

The Gap-Hunter scaffold differs from all of these in its explicit formulation around the absence blindness theory: it is not merely a retry loop but a *presence enforcement mechanism* grounded in the insight that gaps are invisible until externalized.

VIII. DISCUSSION

A. Limitations

Absence blindness is a structural property of the NTP objective, but it is not absolute. Several mechanisms partially counteract it:

- 1) *Instruction following*: Fine-tuning on instruction-following data can encode checklist-like behaviors implicitly, so that the model *implicitly* completes structured tasks. Heo et al. [32] show that internal states encode instruction compliance, suggesting that instruction-tuned models may partially internalize presence manifests. However, this effect is task-specific and does not generalize to novel gap types.
- 2) *Emergent planning*: Dong et al. [33] show that LLM hidden representations encode future output structure beyond the immediate next token, suggesting that deep models implicitly plan ahead. However, this emergent planning operates over probable continuations, not over absent-but-required deliverables—the gap remains invisible even if the model has latent planning capacity.
- 3) *Test-time compute scaling*: Extended chain-of-thought reasoning (e.g., OpenAI o1/o3 models) introduces more generation steps, potentially allowing gaps to become visible as the model articulates its reasoning. However, Heyman and Zylberberg [11] demonstrate that even o3-mini maintains hallucination patterns driven by absent-but-filled edges, suggesting that extended thinking does not resolve absence blindness.

B. Ethical Implications

Absence blindness has direct safety implications for high-stakes LLM deployments. In clinical documentation, Mess et al. [14] warn that “insidious and potentially significant errors of omission” in AI-generated medical notes can harm patients. In legal and financial contexts, an unverified claim that is absent from the model’s epistemic ground may be generated as confident output with no signal of uncertainty. The EPP framework provides an actionable principle for safety engineers: every high-stakes deployment must specify a presence manifest and a gap-hunting mechanism.

C. Theoretical Connection to Formal Languages

The Gap-Hunter scaffold shares structural properties with runtime verification in formal methods [34]: a finite automaton over task states (PENDING, SATISFIED, FAILED) that monitors execution against a specification. LLM agents operating without such a monitor are analogous to programs running without assertions—they may terminate successfully or silently violate invariants with no observable difference at the output level.

IX. CONCLUSION

We have argued that LLMs are structurally blind to absence: the next-token prediction objective confers influence only on tokens that are present in context, and thus the model cannot perceive missing deliverables, unverified claims, or

uncompleted task steps. We named this property absence blindness, formalized it in terms of the NTP loss function, and showed that the entire literature on LLM failure mitigation—from RAG to checklist prompting to verification agents—converges on a single design principle: **gaps must be hunted from the outside and made into present objects.**

The Externalized Presence Principle is not a new technique but a unifying explanation for why existing techniques work. Its practical upshot is a design imperative: any system deploying LLMs in high-stakes agentic settings must include an external gap-hunter that enumerates required deliverables, detects unsatisfied items after each generation step, and injects gap objects back into context until the presence manifest is complete. Absence, left unaddressed, is invisible. Presence, explicitly constructed, is the only force that moves the next token.

REFERENCES

- [1] L. Chen et al., “Next token prediction towards multimodal intelligence: A comprehensive survey,” arXiv:2412.18619, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2412.18619>
- [2] R. T. McCoy, S. Yao, D. Friedman, M. Hardy, and T. L. Griffiths, “Embers of autoregression: Understanding large language models through the problem they are trained to solve,” arXiv:2309.13638, Sep. 2023. [Online]. Available: <https://arxiv.org/abs/2309.13638>
- [3] G. Bachmann and V. Nagarajan, “The pitfalls of next-token prediction,” arXiv:2403.06963, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2403.06963>
- [4] S. M. Downes, P. Forber, and A. Grzankowski, “LLMs are not just next token predictors,” arXiv:2408.04666, Aug. 2024. [Online]. Available: <https://arxiv.org/abs/2408.04666>
- [5] S. Banerjee, A. Agarwal, and S. Singla, “LLMs will always hallucinate, and we need to live with this,” arXiv:2409.05746, Sep. 2024. [Online]. Available: <https://arxiv.org/abs/2409.05746>
- [6] Z. Xu, S. Jain, and M. Kankanhalli, “Hallucination is inevitable: An innate limitation of large language models,” arXiv:2401.11817, Feb. 2025. [Online]. Available: <https://arxiv.org/abs/2401.11817>
- [7] A. Dejl et al., “Comprehensiveness metrics for automatic evaluation of factual recall in text generation,” arXiv:2510.07926, Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2510.07926>
- [8] N. F. Liu et al., “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, 2024. DOI: https://doi.org/10.1162/tacl_a_00638
- [9] C.-Y. Hsieh et al., “Found in the middle: Calibrating positional attention bias improves long context utilization,” arXiv:2406.16008, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2406.16008>
- [10] T. Lu, M. Gao, K. Yu, A. Byerly, and D. Khashabi, “Insights into LLM long-context failures: When transformers know but don’t tell,” arXiv:2406.14673, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2406.14673>
- [11] A. Heyman and J. Zylberberg, “Reasoning large language model errors arise from hallucinating critical problem features,” arXiv:2505.12151, May 2025. [Online]. Available: <https://arxiv.org/abs/2505.12151>
- [12] M. Omar et al., “Multi-model assurance analysis showing large language models are highly vulnerable to adversarial hallucination attacks during clinical decision support,” *Commun. Med.*, 2025. DOI: <https://doi.org/10.1038/s43856-025-01021-3>
- [13] H. Li, H. Chi, M. Liu, and W. Yang, “Look within, why LLMs hallucinate: A causal perspective,” arXiv:2407.10153, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2407.10153>
- [14] S. A. Mess, A. Mackey, and D. E. Yarowsky, “Artificial intelligence scribe and large language model technology in healthcare documentation: Advantages, limitations, and recommendations,” *Plast. Reconstr. Surg. Global Open*, vol. 13, Jan. 2025. DOI: <https://doi.org/10.1097/GOX.00000000000006450>
- [15] K. Lee, E. Kim, J. Choi, and B. Chang, “NOAH: Benchmarking narrative prior driven hallucination and omission in video large language models,” arXiv:2511.06475, Nov. 2025. [Online]. Available: <https://arxiv.org/abs/2511.06475>

- [16] R. Kamoi et al., “When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs,” *Trans. Assoc. Comput. Linguistics*, Jun. 2024. DOI: https://doi.org/10.1162/tacl_a_00713
- [17] K. Chen, F.-Y. Su, and J.-H. Chiang, “The self-correction illusion: LLMs correct others but not themselves,” Semantic Scholar, Jun. 2026. [Online]. Available: <https://www.semanticscholar.org/paper/3d637e18c5dad37d347dff3b45149ce37081b2d>
- [18] L. Pan et al., “Automatically correcting large language models: Surveying the landscape of diverse self-correction strategies,” arXiv:2308.03188, Aug. 2023. [Online]. Available: <https://arxiv.org/abs/2308.03188>
- [19] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [20] Y. Zhang, “A retrieval-augmented generation framework with retriever and generator modules for enhancing factual consistency,” *Applied and Computational Engineering*, Jul. 2025. DOI: <https://doi.org/10.54254/2755-2721/2025.tj24496>
- [21] M. C. Wood and A. A. Forbes, “100% elimination of hallucinations on RAGTruth for GPT-4 and GPT-3.5 Turbo,” arXiv:2412.05223, Mar. 2025. [Online]. Available: <https://arxiv.org/abs/2412.05223>
- [22] Y.-K. Liu and Y.-C. Tsai, “Quality-driven agentic reasoning for LLM-assisted software design: Questions-of-Thoughts (QoT) as a time-series self-QA chain,” arXiv:2603.11082, Mar. 2026. [Online]. Available: <https://arxiv.org/abs/2603.11082>
- [23] B. He et al., “Retrieving, rethinking and revising: The chain-of-verification can improve retrieval augmented generation,” arXiv:2410.05801, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2410.05801>
- [24] G. Kim, P. Baldi, and S. McAleer, “Language models can solve computer tasks,” arXiv:2303.17491, Nov. 2023. [Online]. Available: <https://arxiv.org/abs/2303.17491>
- [25] J. Sun, S. Y. Min, Y. Chang, and Y. Bisk, “Tools fail: Detecting silent errors in faulty tools,” arXiv:2406.19228, Jun. 2024. [Online]. Available: <https://arxiv.org/abs/2406.19228>
- [26] Y. Qu, T. Zhang, N. Garg, and A. Kumar, “Recursive introspection: Teaching language model agents how to self-improve,” arXiv:2407.18219, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2407.18219>
- [27] M. Renze and E. Guven, “Self-reflection in LLM agents: Effects on problem-solving performance,” arXiv:2405.06682, Oct. 2024. [Online]. Available: <https://arxiv.org/abs/2405.06682>
- [28] J. Zhang et al., “AFlow: Automating agentic workflow generation,” arXiv:2410.10762, Feb. 2025. [Online]. Available: <https://arxiv.org/abs/2410.10762>
- [29] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” arXiv:2503.18666, Apr. 2025. [Online]. Available: <https://arxiv.org/abs/2503.18666>
- [30] Y. Hu et al., “QualityFlow: An agentic workflow for program synthesis controlled by LLM quality checks,” arXiv:2501.17167, Mar. 2025. [Online]. Available: <https://arxiv.org/abs/2501.17167>
- [31] J. C. M. Tan et al., “TaskGen: A task-based, memory-infused agentic framework using StrictJSON,” arXiv:2407.15734, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2407.15734>
- [32] J. Heo et al., “Do LLMs ‘know’ internally when they follow instructions?” arXiv:2410.14516, Mar. 2025. [Online]. Available: <https://arxiv.org/abs/2410.14516>
- [33] Z. Dong et al., “Emergent response planning in LLM,” arXiv:2502.06258, Feb. 2025. [Online]. Available: <https://arxiv.org/abs/2502.06258>
- [34] Z. Li et al., “Formal-LLM: Integrating formal language and natural language for controllable LLM-based agents,” arXiv:2402.00798, Aug. 2024. [Online]. Available: <https://arxiv.org/abs/2402.00798>